

Addis Ababa University
College of Natural and Computational Science
Department of Physics

Computational Physics Application

(Phys 4624)

Exercise: Numerical Simulation of Linear and Non-linear Pendulum

Name: Getabalew Manegerew
ID Number: UGR/6981/15
Instructor: Dr. Lemi D.

March 30, 2026

March 30, 2026

1 Introduction

The equation of motion for a simple pendulum is given by:

$$\frac{d^2\theta}{dt^2} + \frac{g}{\ell} \sin \theta = 0 \quad (1)$$

For small angles, we use the approximation $\sin \theta \approx \theta$, leading to Simple Harmonic Motion (SHM). In this report, we compare numerical solutions using the **Euler Method** and the **Fourth-Order Runge-Kutta (RK4)** method.

2 Exercise I: Small Amplitude Approximation

Using $g/\ell = 9$, $\theta(0) = 0.1$, and $\omega(0) = 0$, we solve the linear equation:

$$\frac{d^2\theta}{dt^2} + 9\theta = 0$$

```
1 PROGRAM pendulum
2   IMPLICIT NONE
3   REAL, EXTERNAL :: alpha
4   REAL, PARAMETER :: dt=0.01, t0=0, tf=20, theta0=0.1, omega0=0 !here
the parameters and initial conditions are sat
5   INTEGER, PARAMETER :: N=(tf-t0)/dt !this is used to declare the array
size of the angular velocity and the time
6   REAL :: theta(N), omega(N), t(N), theta_k(N), omega_k(N), t_k(N) !here
angular displacement, angular velocity and time are declared an array
of N size
7   REAL :: period_e, period_k, dl_e, dl_k
8   INTEGER :: i, k
9   LOGICAL :: cond1=.False., cond2=.False.
10
11 !below initial conditions are explicitly sat for EULER method
12 t(1) = t0
13 theta(1) = theta0
14 omega(1) = omega0
15
16 !below initial conditions are explicitly sat for RK4 method
17 t_k(1) = t0
18 theta_k(1) = theta0
19 omega_k(1) = omega0
20
```

```

21      !this loop calls EULER and RK4 subroutines and updates the arrays
      value
22      DO i=1, N-1
23          CALL EULER(t(i), t(i+1), dt, theta(i), theta(i+1), omega(i),
      omega(i+1))
24          CALL RK(t_k(i), t_k(i+1), dt, theta_k(i), theta_k(i+1),
      omega_k(i), omega_k(i+1))
25
26          !belows parameters are useful to find a match for initial
      condition of the pendulum for each method
27          dl_e = theta0 - theta(2)
28          dl_k = theta0 - theta_k(2)
29
30          !this conditional statement check when the EULER meet initial
      setup
31          IF (Abs(theta(i+1)-theta0)<dl_e.and.(.not.cond1)) THEN
32              period_e = t(i+1)
33              cond1 = .True.
34          END IF
35
36          !this conditional statement check when the RK4 meet initial
      setup
37          IF (Abs(theta_k(i+1)-theta0)<dl_k.and.(.not.cond2)) THEN
38              period_k = t_k(i+1)
39              cond2 = .True.
40          END IF
41
42          !this is to exit the loop when the period for both methods
      reached
43          IF (cond1.and.cond2) EXIT
44
45      END DO
46
47
48      WRITE(*,*) "The period in EULER method: p_e =", period_e
49      WRITE(*,*) "The period in RK4 method: p_k =", period_k
50      WRITE(*,*) "The the difference is: dT=", period_k - period_e
51
52  END PROGRAM pendulum
53
54  !this function is for angular acceleration
55  FUNCTION alpha(omega, theta, t) RESULT(y)
56      IMPLICIT NONE
57      REAL :: omega, theta, t, y, g=9.8, l=9.8/9
58      y = -g*theta/l
59  END FUNCTION alpha
60
61  !defining subrouting for EULER method
62  SUBROUTINE euler(t_, t, dt, theta_, theta, omega_, omega)
63      IMPLICIT NONE
64      REAL, EXTERNAL :: alpha
65      REAL :: t_, t, dt, theta_, theta, omega_, omega, a !underscore "_" is
      used for the ith element while the rest are for i+1 th
66

```

```

67     t = t_ + dt
68     a = alpha(omega_, theta_, t_)
69     omega = omega_ + a*dt
70     theta = theta_ + omega*dt
71
72 END SUBROUTINE euler
73
74 !defining subrouting for RK4 method
75 SUBROUTINE rk(t_, t, dt, theta_, theta, omega_, omega)
76     IMPLICIT NONE
77     REAL, EXTERNAL :: alpha
78     REAL :: t_, t, dt, theta_, theta, omega_, omega !underscore "_" is
79     used for the ith element while the rest are for i+1 th
80     REAL :: k1, k2, k3, k4, p1, p2, p3, p4 !k's are small increments in
81     angular velocity (omega), while p's are for angular displacement (theta
82     )
83
84     t = t_ + dt
85     p1 = omega_*dt
86     k1 = alpha(omega_, theta_, t_)*dt
87
88     k2 = alpha(omega_+k1/2, theta_+p1/2, t_+dt/2)*dt
89     p2 = (omega_ + k1/2)*dt
90
91     k3 = alpha(omega_+k2/2, theta_+p2/2, t_+dt/2)*dt
92     p3 = (omega_ + k2/2)*dt
93
94     k4 = alpha(omega_+k3, theta_+p3, t_+dt)*dt
95     p4 = (omega_ + k3)*dt
96
97     theta = theta_ + (p1 + 2*(p2 + p3) + p4)/6
98     omega = omega_ + (k1 + 2*(k2 + k3) + k4)/6
99
100 END SUBROUTINE rk
101 \label{ls:period}

```

Listing 1: Fortran 90 implementation of Pendulum Motion

2.1 Simulation Output

The period in EULER method: $p_e = 2.07999849$
 The period in RK4 method: $p_k = 2.08999848$
 The difference is: $dT = 9.99999046 \times 10^{-3}$

3 Exercise II: Large Amplitude and Energy Stability

To simulate large amplitudes, we replace θ with $\sin \theta$. We monitor the total energy E to ensure stability:

$$E = \frac{1}{2}m\ell^2 \left(\frac{d\theta}{dt} \right)^2 + mg\ell(1 - \cos \theta) \quad (2)$$

Without affecting the trend, we set the mass $m = 1kg$. For a stable solution, the energy should remain constant over time. As it can be inferred from figure ??, RK4 marked a significant stability over EULER method.

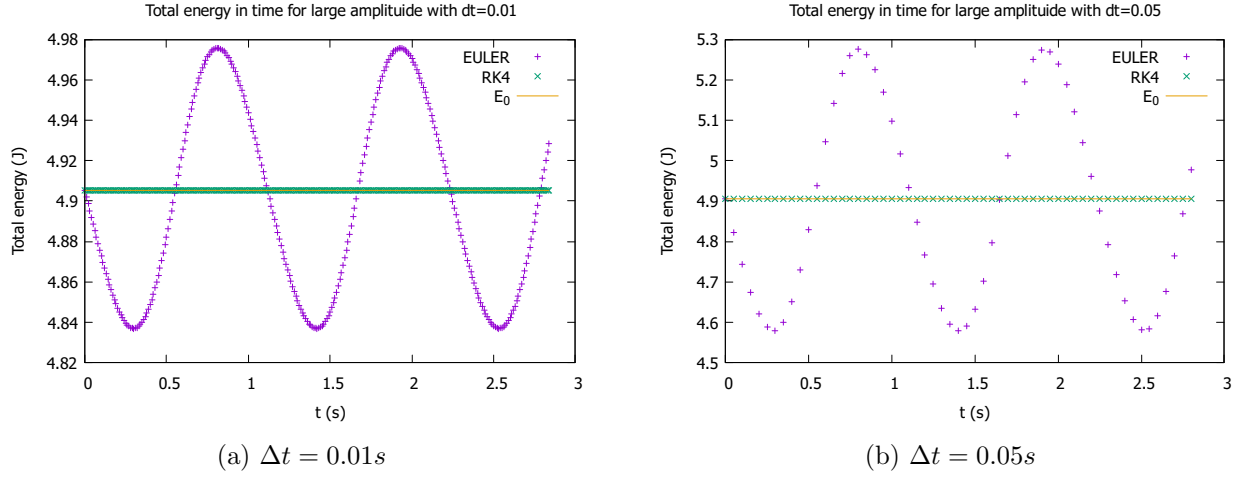


Figure 1: Comparison of Pendulum Total energy (E_0) over time. Note the energy drift in the Euler method versus the stability of RK4.

4 Exercise III: Period vs. Amplitude

I vary the initial condition $\theta(0)$ from 0.1 to 1.0 radians. And in this part of the analysis of the problem, I implemented only the robust RK4 with a time step $dt = 0.01s$.

4.1 Results and Discussion

As the amplitude increases, the linear approximation $\sin \theta \approx \theta$ fails. The restoring force in the non-linear case is effectively "weaker" than the linear case at high angles, resulting in a **larger period** for the non-linear pendulum.

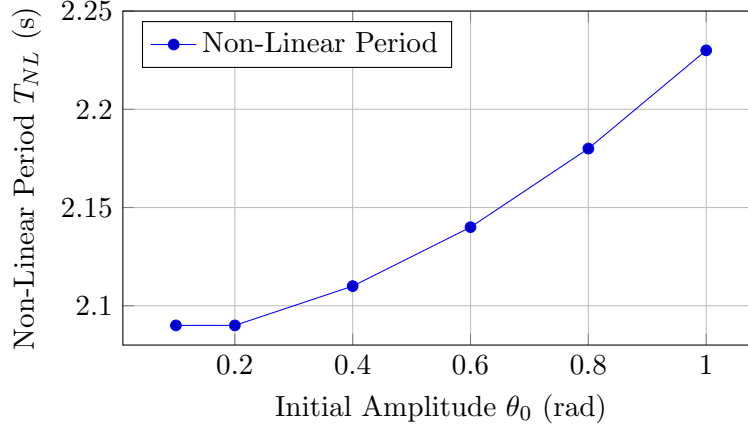


Figure 2: Variation of Non-Linear Pendulum Period with Initial Amplitude ($g/\ell = 9.0$)

The comparison of the linear period T_L and Non-linear period T_{NL} are displayed in the table 1. As can be inferred from the table, linear period remains constant, while non-linear period increases as the amplitude does.

Table 1: Comparison of Period T for Linear vs. Non-Linear Pendulum ($g/\ell = 9.0$)

Initial Amplitude θ_0 (rad)	Linear Period T_L (s)	Non-Linear Period T_{NL} (s)	Difference ΔT (%)
0.1	2.08999848	2.08999848	0.00%
0.2	2.08999848	2.08999848	0.00%
0.4	2.08999848	2.10999846	0.96%
0.6	2.08999848	2.13999844	2.39%
0.8	2.08999848	2.17999840	4.29%
1.0	2.08999848	2.22999835	6.70%

5 Fortran Code Implementation

```

1 PROGRAM pendulum
2   IMPLICIT NONE
3   REAL, EXTERNAL :: alpha
4   REAL, PARAMETER :: dt=0.01, t0=0, tf=20, theta0=0.1, omega0=0, g=9.8,
5   l=9.8/9 !here the parameters and initial conditions are sat
6   INTEGER, PARAMETER :: N=(tf-t0)/dt !this is used to declare the array
7   size of the angular velocity and the time
8   REAL :: theta(N), omega(N), t(N), theta_k(N), omega_k(N), t_k(N) !here
9   angular displacement, angular velocity and time are declared an array
10  of N size
11  REAL :: energy_initial, energy_eu(N), energy_rk(N)
12  INTEGER :: i, k
13
14  !below initial conditions are explicitly sat for EULER method
15  t(1) = t0
16  theta(1) = theta0

```

```

14     omega(1) = omega0
15
16     !below initial conditions are explicitly sat for RK4 method
17     t_k(1) = t0
18     theta_k(1) = theta0
19     omega_k(1) = omega0
20
21     !this loop calls EULER and RK4 subroutines and updates the arrays
    value
22     DO i=1, N-1
23
24         CALL EULER(t(i), t(i+1), dt, theta(i), theta(i+1), omega(i),
    omega(i+1))
25         CALL RK(t_k(i), t_k(i+1), dt, theta_k(i), theta_k(i+1),
    omega_k(i), omega_k(i+1))
26
27     END DO
28
29     energy_initial = 0.5*(l*omega0)**2 + g*l*(1-Cos(theta0))
30     energy_eu = 0.5*(l*omega)**2 + g*l*(1-Cos(theta))
31     energy_rk = 0.5*(l*omega_k)**2 + g*l*(1-Cos(theta_k))
32
33
34     OPEN(UNIT=8, FILE="pendulum.csv", POSITION="APPEND", ACCESS="SEQUENTIAL
    ", STATUS="REPLACE")
35     DO i=1, N
36         WRITE(UNIT=8, FMT="(5F18.10)") t(i), theta(i), omega(i),
    theta_k(i), omega_k(i), energy_eu(i), energy_rk(i)
37     END DO
38     CLOSE(UNIT=8)
39
40
41
42 END PROGRAM pendulum
43
44 !this function is for angular acceleration
45 FUNCTION alpha(omega, theta, t) RESULT(y)
46     IMPLICIT NONE
47     REAL :: omega, theta, t, y, g=9.8, l=9.8/9
48     y = -g*Sin(theta)/l
49 END FUNCTION alpha
50
51 !defining subrouting for EULER method
52 SUBROUTINE euler(t_, t, dt, theta_, theta, omega_, omega)
53     IMPLICIT NONE
54     REAL, EXTERNAL :: alpha
55     REAL :: t_, t, dt, theta_, theta, omega_, omega, a !underscore "_" is
    used for the ith element while the rest are for i+1 th
56
57     t = t_ + dt
58     a = alpha(omega_, theta_, t_)
59     omega = omega_ + a*dt
60     theta = theta_ + omega*dt
61

```

```

62 END SUBROUTINE euler
63
64 !defining subrouting for RK4 method
65 SUBROUTINE rk(t_, t, dt, theta_, theta, omega_, omega)
66     IMPLICIT NONE
67     REAL, EXTERNAL :: alpha
68     REAL :: t_, t, dt, theta_, theta, omega_, omega !underscore "_" is
69     used for the ith element while the rest are for i+1 th
70     REAL :: k1, k2, k3, k4, p1, p2, p3, p4 !k's are small increments in
71     angular velocity (omega), while p's are for angular displacement (theta
72     )
73
74     t = t_ + dt
75     p1 = omega_*dt
76     k1 = alpha(omega_, theta_, t_)*dt
77
78     k2 = alpha(omega_+k1/2, theta_+p1/2, t_+dt/2)*dt
79     p2 = (omega_ + k1/2)*dt
80
81     k3 = alpha(omega_+k2/2, theta_+p2/2, t_+dt/2)*dt
82     p3 = (omega_ + k2/2)*dt
83
84     k4 = alpha(omega_+k3, theta_+p3, t_+dt)*dt
85     p4 = (omega_ + k3)*dt
86
87     theta = theta_ + (p1 + 2*(p2 + p3) + p4)/6
88     omega = omega_ + (k1 + 2*(k2 + k3) + k4)/6
89
90 END SUBROUTINE rk

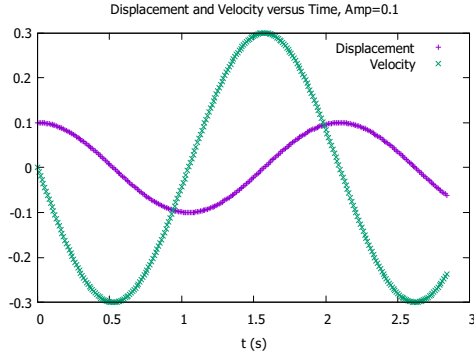
```

Listing 2: Fortran 90 implementation of Pendulum Motion

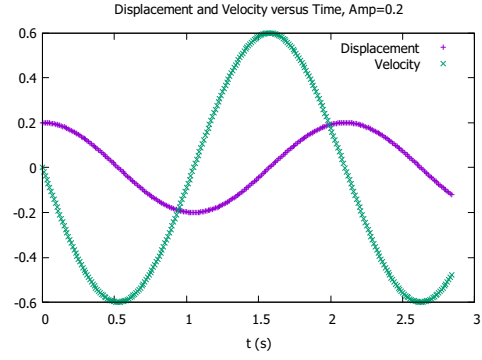
6 Visualizations

To plot the graphs, I used a time range up to *3seconds*, in order to keep the trend visibility. Additionally, as stated previously, the simulation is based on RK4.

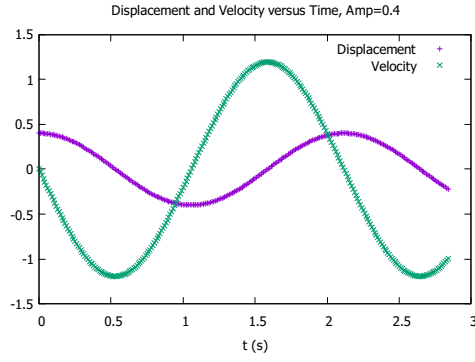
Figure 3: The plot of the angular displacement and angular velocity vs time.



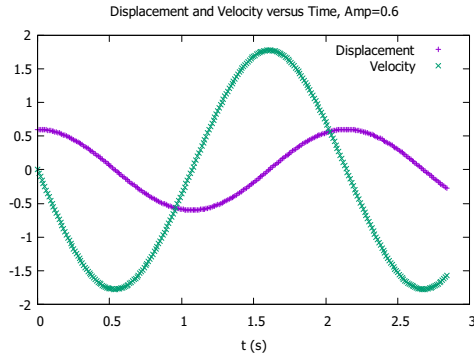
(a) Amp = 0.1



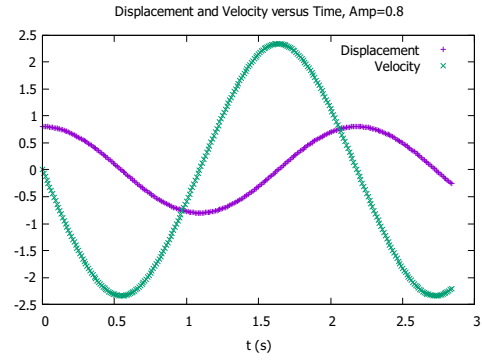
(b) Amp = 0.2



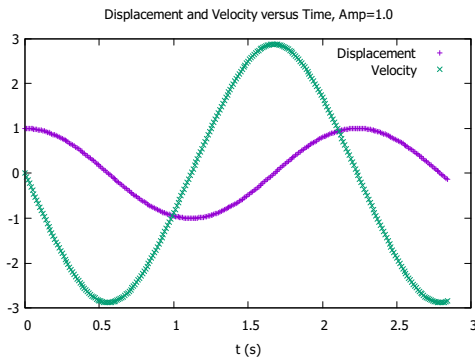
(c) Amp = 0.4



(d) Amp = 0.6



(e) Amp = 0.8



(f) Amp = 1.0